Lab 04: Modes of Operation

Overview

In this lab you will implement encryption/decryption systems for the five modes of operation discussed in the lecture, using AES as the block cipher: Electronic Code Book (ECB), Cipher Block Chaining (CBC), Output Feedback (OFB), Cipher Feedback (CFB) and Counter mode (CTR). In all modes except ECB the 16-byte encryption IV is chosen at random and is prepended to the ciphertext.

For ECB encryption we use the 10…0 padding and for CBC encryption the PKCS5 padding scheme discussed in the lecture; OFB, CFB and CTR need no padding. While we ask that you implement both encryption and decryption, we will only test the decryption function. In the following questions you are given an AES key and a ciphertext (both are hex encoded) and your goal is to recover the plaintext.

For an implementation of AES you may use an existing crypto library such as PyCrypto or PyCryptodome (Python), Crypto++ (C++), or any other. While it is fine to use the built-in AES functions, we ask that as a learning experience you implement the five modes yourself.

You must submit the source code of the program you wrote to decode.

Background

As in the lecture, Enc_k denotes AES under key k on a single 16-byte block and Dec_k its inverse. The plaintext is split into 16-byte blocks $x_0, x_1, \ldots, x_n$ and the ciphertext blocks are $y_0, y_1, \cdots, y_n$.

The IV is part of the ciphertext. In CBC, OFB, CFB and CTR the encryptor picks a fresh random 16-byte IV, uses it, and *writes it in the clear in front of the ciphertext blocks*:

$$\underbrace{c}_{\text{ciphertext}} = \underbrace{IV}_{\text{16 bytes}} \| y_0 \| y_1 \| \cdots \| y_n.$$

The IV is not secret. The decryptor must first split off the leading 16 bytes (the first 32 hex digits) as the IV. ECB uses no IV: its ciphertext is just $y_0 \| \cdots \| y_n$. In Figures 1–6, the dashed frames show exactly what makes up the *plaintext* and the *ciphertext*.

ECB mode

Encryption: $y_i = \text{Enc}_k(x_i)$ for $i = 0, 1, \ldots, n$. Decryption: $x_i = \text{Dec}_k(y_i)$, then check and remove the 10…0 padding (Figure 1).

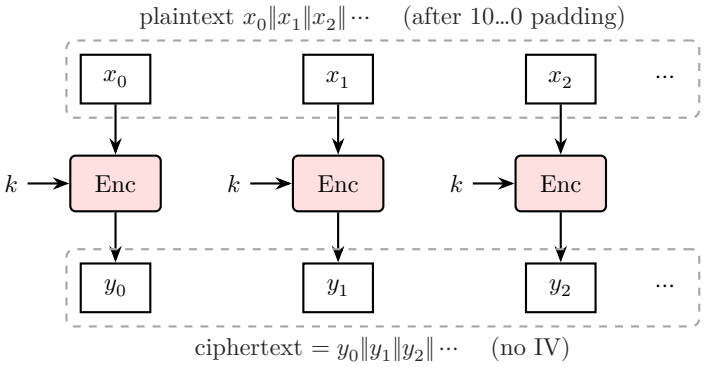


Figure 1: ECB encryption. Each block is encrypted independently; decryption applies Dec_k to each block.

ECB is deterministic: if $x_i = x_j$ then $y_i = y_j$, so patterns in the plaintext remain visible in the ciphertext. It is included only for its historical significance and should never be used in practice.

CBC mode

Encryption: choose a random IV and set $y_{-1} = IV$; then $y_i = \text{Enc}_k(x_i \oplus y_{i-1})$ for $i = 0, 1, \ldots, n$. Output $IV \| y_0 \| \cdots \| y_n$ (Figure 2).

Decryption: take the *first 16 bytes* of the ciphertext as $y_{-1} = IV$; then $x_i = \text{Dec}_k(y_i) \oplus y_{i-1}$; finally check and remove the PKCS5 padding (Figure 3).

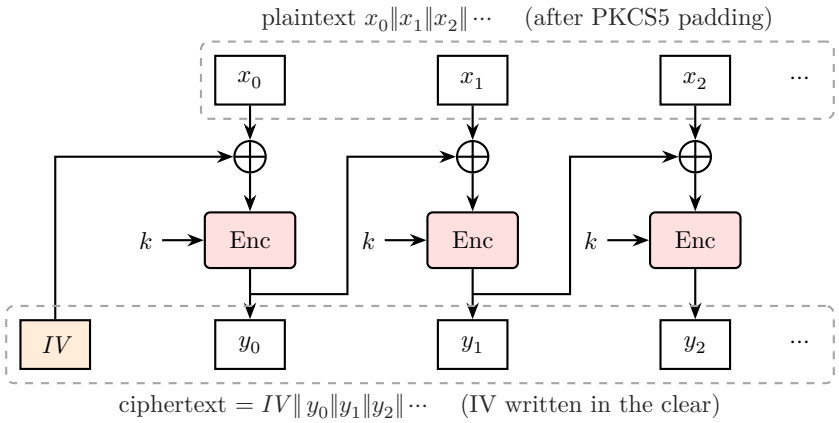


Figure 2: CBC encryption. The random IV is XORed into the first block and written in the clear in front of the ciphertext blocks.

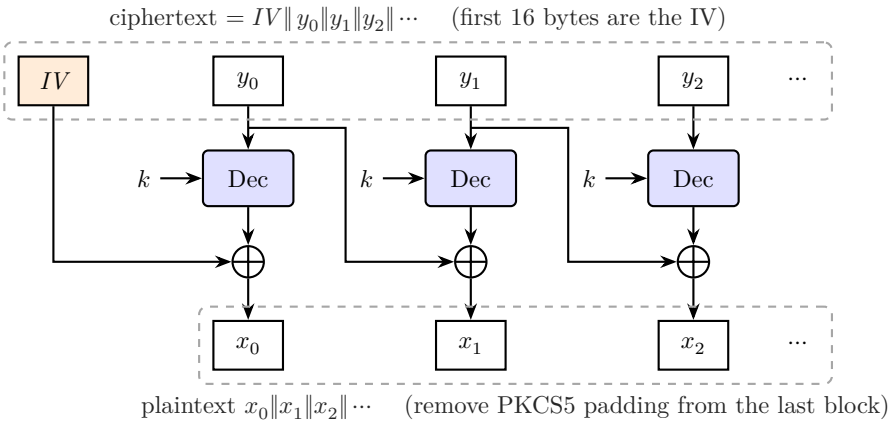


Figure 3: CBC decryption. The first 16 bytes of the ciphertext are the IV; the PKCS5 padding is checked and removed from the last plaintext block.

Padding

ECB and CBC can only encrypt a whole number of blocks, so the plaintext is padded before encryption. Padding is *always* added, so that it can be removed without ambiguity: if the plaintext length is already a multiple of 16, a full dummy block is appended. If b bytes ($1 \leq b \leq 16$) are missing:

- *PKCS5* (CBC): append b bytes, each of value b. (For 16-byte blocks this scheme is also known as PKCS#7.)
- *10…0* (ECB): append one byte `0x80` (the bit string 1000 0000) followed by $b-1$ zero bytes.

Bytes missing b	PKCS5 (CBC)	10…0 (ECB)
1	`01`	`80`
2	`02 02`	`80 00`
5	`05 05 05 05 05`	`80 00 00 00 00`
16	`10 10 ... 10`	`80 00 ... 00`

Decryption must check the padding and reject the ciphertext if the padding is invalid.

OFB mode

Encryption: choose a random IV, set $s_{-1} = IV$ and compute the keystream $s_i = \mathrm{Enc}_k(s_{i-1})$; then $y_i = x_i \oplus s_i$ for $i = 0, 1, \ldots, n$. Output $IV \| y_0 \| \cdots \| y_n$ (Figure 4). Decryption takes the IV from the ciphertext, rebuilds the same keystream and computes $x_i = y_i \oplus s_i$.

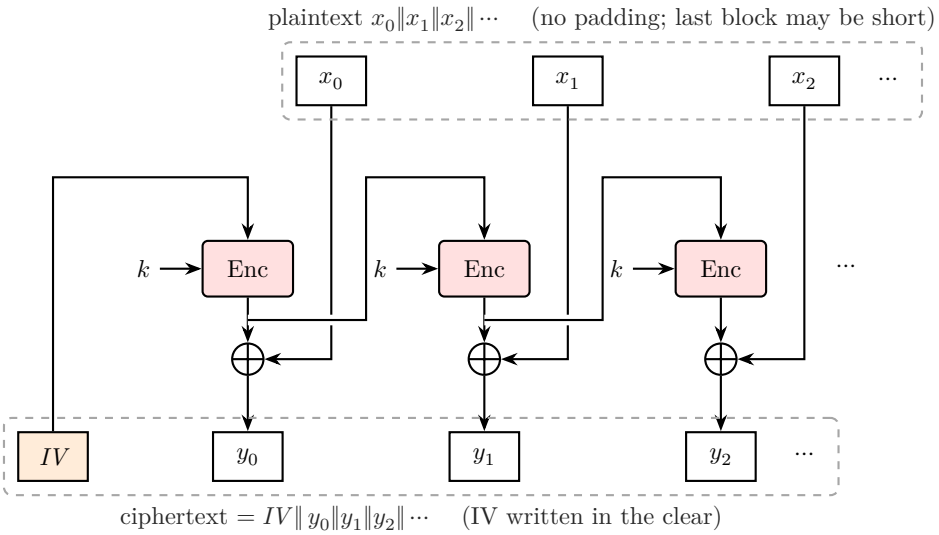


Figure 4: OFB encryption. The output of each Enc is fed back into the next Enc; the keystream does not depend on the plaintext.

CFB mode

Encryption: choose a random IV and set $y_{-1} = IV$; then $y_i = x_i \oplus \mathrm{Enc}_k(y_{i-1})$ for $i = 0, 1, \ldots, n$. Output $IV \| y_0 \| \cdots \| y_n$ (Figure 5). Working out the decryption is part of the lab (see the exercise in the lecture).

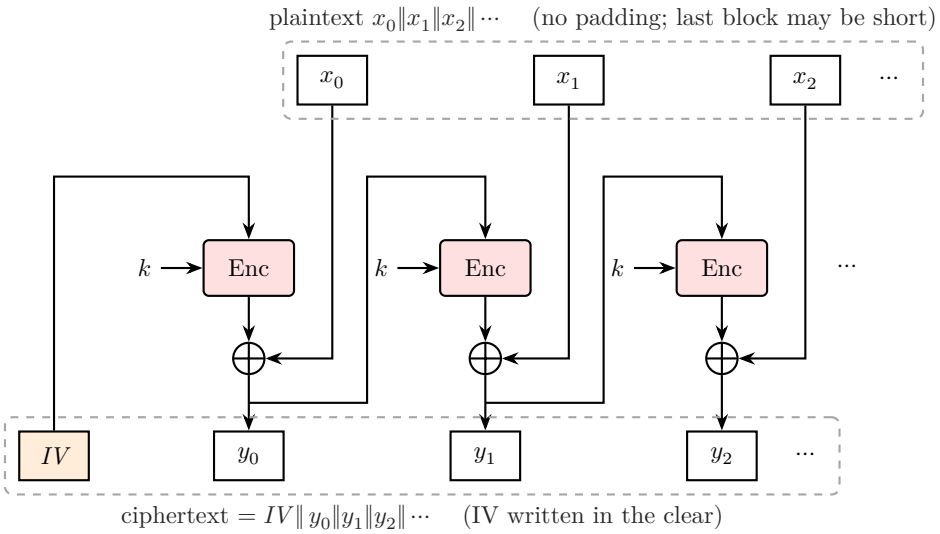


Figure 5: CFB encryption. Each ciphertext block is fed back into the next Enc.

CTR mode

Encryption: choose a random IV, treated as a 128-bit big-endian integer; then $y_i = x_i \oplus \mathrm{Enc}_k(IV + i \bmod 2^{128})$ for $i = 0, 1, \ldots, n$. Output $IV \| y_0 \| \cdots \| y_n$ (Figure 6). Working out the decryption is part of the lab (see the exercise in the lecture).

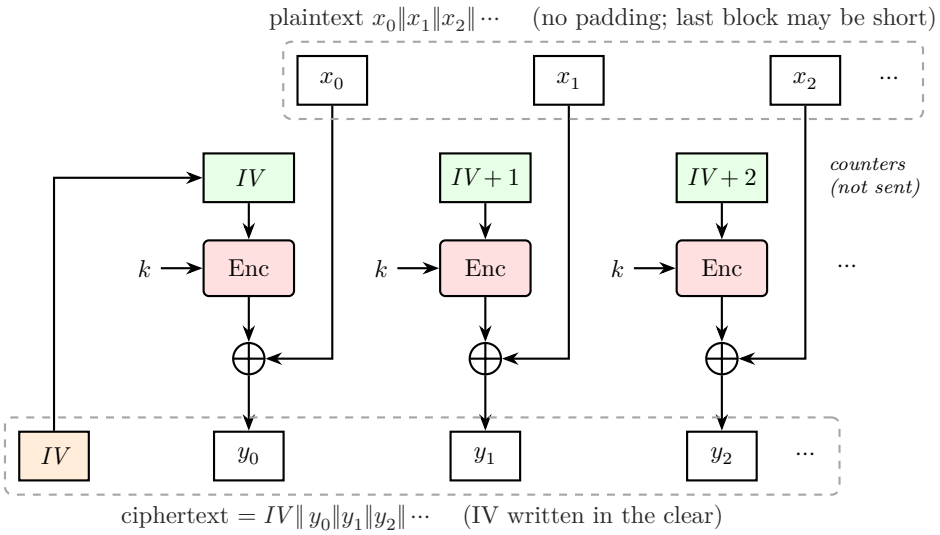


Figure 6: CTR encryption. The IV is the first counter and is written in the clear in front of the ciphertext blocks.

In OFB, CFB and CTR no padding is used: the last block x_n may be shorter than 16 bytes

and is XORed with only as many bytes of $\text{Enc}_k(\cdot)$ as it has, so the ciphertext is exactly 16 bytes (the IV) longer than the plaintext.

Data format

Each ciphertext is split over several lines for readability; concatenate the lines (with no spaces) before hex-decoding. The same data is provided in machine-readable form in `lab04_questions.txt` (one question per line: number, mode, key, ciphertext); reading it from this file avoids copying errors. Except for ECB, every ciphertext *includes the IV*: its first 32 hex digits (16 bytes) are the IV, and the rest are the encrypted blocks $y_0, y_1, ...$

Question 1

ECB key: `c59c48f7fc27bb1fa669ccd55c152015`

ECB Ciphertext 1:

```
a1dc4d1fe237b8ea72bb49948cca4861041d1a5469fc685d330378acebbdb800
a1dc4d1fe237b8ea72bb49948cca4861cce62b9b08f9a81fd1541d6a0136be21
```

What is the plaintext? Compare the ciphertext blocks y_0 and y_2.

Question 2

ECB key: `c59c48f7fc27bb1fa669ccd55c152015`

ECB Ciphertext 2:

```
0734805f750988ab9ddd6102feb457ba
```

What is the plaintext?

Question 3

ECB key: `c59c48f7fc27bb1fa669ccd55c152015`

ECB Ciphertext 3:

```
c14d8d450222bf44575e49f310327322044d2f72cc15ec55e94546c8251f4124
0f708833189640d9dc8811d08ebbecd2
```

What is the plaintext?

Question 4

CBC key: `140b41b22a29beb4061bda66b6747e14`

CBC Ciphertext 1:

```
4ca00ff4c898d61e1edbf1800618fb2828a226d160dad07883d04e008a7897ee
2e4b7465d5290d0c0e6c6822236e1daafb94ffe0c5da05d9476be028ad7c1d81
```

What is the plaintext?

Question 5

CBC key: `140b41b22a29beb4061bda66b6747e14`

CBC Ciphertext 2:

```
5b68629feb8606f9a6667670b75b38a5b4832d0f26e1ab7da33249de7d4afc48
e713ac646ace36e872ad5fb8a512428a6e21364b0c374df45503473c5242a253
```

What is the plaintext?

Question 6

CBC key: `e7f3bd5bb02ff3ed672564e119b8bd5e`

CBC Ciphertext 3:

```
b750fc2728c3c16776fa197c5d49d46f83294479e48fd5e753e13106415a6185
76000e060f7fe33e309f1aa26001a4ce8f39fe8669a8c8db459bced1174e5e45
```

What is the plaintext?

Question 7

CBC key: `e7f3bd5bb02ff3ed672564e119b8bd5e`

CBC Ciphertext 4:

```
8935980af373f8746705a0f332570f6653179a65d8d191c3254fe0004dd92c10
```

What is the plaintext?

Question 8

OFB key: `01a1babc3cbfc48932761127997cdc40`

OFB Ciphertext 1:

```
423b1d6d0d11d3fab1236e2e03294345eaa47185fedf20251c5c1e2bcb3ef400
35003d37cdd5cae757a2d59483437a279fa215ee7d0045596bd0474c981c76fd
47c946be17c4e91238ff
```

What is the plaintext?

Question 9

OFB key: `01a1babc3cbfc48932761127997cdc40`

OFB Ciphertext 2:

```
bbc9645adb9a7020010481aa0bfcde491a4ff192b902dc63f5b44ce05cce36ff
dbe35a2b779e46ca10bd773abfea383ab89afb7d9b5667dbc6be72adc15558
```

What is the plaintext?

Question 10

CFB key: `418d6b6d8bf2f8a873509336c64cbb4d`

CFB Ciphertext 1:

```
0e31aac86c0532fde5e69aa86e6d2c3ecb9fce6486b716e54cc60c396e31f7b1
3feed1aa150ac46ad65bd7540e55a5e602a206a7a42007d4663c82ad281f21cb
f59f9127e2f2b5
```

What is the plaintext?

Question 11

CFB key: `418d6b6d8bf2f8a873509336c64cbb4d`

CFB Ciphertext 2:

```
3debb9636423ee952ee93717209eb264595366f2ca9ab6c73536624ab717d119
7a7518a6a945e3d2b280585f156a6090e725a6eef315ab252a3b23557231b039
66c4f19dd8155e4d8f
```

What is the plaintext?

Question 12

CTR key: `36f18357be4dbd77f050515c73fcf9f2`

CTR Ciphertext 1:

```
69dda8455c7dd4254bf353b773304eec0ec7702330098ce7f7520d1cbbb20fc3
88d1b0adb5054dbd7370849dbf0b88d393f252e764f1f5f7ad97ef79d59ce29f
5f51eeca32eabedd9afa9329
```

What is the plaintext?

Question 13

CTR key: `36f18357be4dbd77f050515c73fcf9f2`

CTR Ciphertext 2:

```
770b80259ec33beb2561358a9f2dc617e46218c0a53cbeca695ae45faa8952aa
0e311bde9d4e01726d3184c34451
```

What is the plaintext?

Question 14

CTR key: `66341ce0412db5c6d90712e055a3b8c6`

CTR Ciphertext 3:

```
10adedeb5d4e5264ffffffffffffffdeb48463612a0a7764c1434dd7b57b5d7
40f974dc2ceae4be9cf2b9011621596dd67844d0e4189d1b177f022baadaa16b
850fd53bd3454a7b47ae30b56ea35962ca956e
```

What is the plaintext?

Question 15

CTR key: `66341ce0412db5c6d90712e055a3b8c6`

CTR Ciphertext 4:

```
ffffffffffffffffffffffffffffffee9ff9dcc6eeb5317b03dc5deae2b18ba
61cf47788632076829a111f416c377a38a5c
```

What is the plaintext?

Submission

Each group submits one file `GroupXX_Lab04.zip` (`XX` = group number) containing the source code and a `README` with the command to run it. The program must print the plaintexts of Questions 1–15. All five modes and both padding schemes must be implemented by hand; a library may be used only for single-block AES.

Grading

Item	Points
Correct plaintexts, Questions 1–15 (2 points each)	30
ECB implementation, including 10…0 padding	10
CBC implementation, including PKCS5 padding	15
OFB implementation	15
CFB implementation	15
CTR implementation	15

Each implementation score covers both encryption and decryption: a fresh random IV prepended on encryption (all modes except ECB), invalid padding rejected (ECB, CBC), a short last block handled without padding (OFB, CFB, CTR) and a full 128-bit counter (CTR). Using a library's built-in mode gives 0 points for that mode.